

**Global availability:** By automatically replicating data across multiple servers, data centres or cloud resources, distributed NoSQL databases can minimise latency and ensure a consistent application experience wherever users are located. An added benefit is a significantly reduced database management burden of manual RDBMS configuration, freeing operations teams to focus on other business priorities.

## Types of NoSQL databases

Several different varieties of NoSQL databases have been created to support specific needs and use cases. These databases can broadly be categorised into four types.

### Key-value store NoSQL database

From an API perspective, key-value stores are the simplest NoSQL data stores to use. The client can either get the value for the key, assign a value for a key or delete a key from the data store. The value is a blob that the data store just stores, without caring or knowing what's inside; it's the responsibility of the application to understand what was stored. Since key-value stores always use primary-key access, they generally have great performance and can be easily scaled. The key-value database uses a hash table to store unique keys and pointers (in some databases it's also called the inverted index) with respect to each data value it stores. There are no column type relations in the database; hence, its implementation is easy. Key-value databases give great performance and can be very easily scaled as per business needs.

**Use cases:** Here are some popular use cases of the key-value databases:

- For storing user session data
- Maintaining schema-less user profiles
- Storing user preferences
- Storing shopping cart data

However key-value databases are not the ideal choice for every use case when:

- We have to query the database by specific data value.
- We need relationships between data values.
- We need to operate on multiple unique keys.

- Our business needs updating a part of the value frequently. Examples of this database are Redis, MemcachedDB and Riak.

### Document store NoSQL database

Document store NoSQL databases are similar to key-value databases in that there's a key and a value. Data is stored as a value. Its associated key is the unique identifier for that value. The difference is that, in a document database, the value contains structured or semi-structured data. This structured/semi-structured value is referred to as a document and can be in XML, JSON or BSON format.

**Use cases:** Document store databases are preferable for:

- E-commerce platforms
- Content management systems
- Analytics platforms
- Blogging platforms

Document store NoSQL databases are not the right choice if you have to run complex search queries or if your application requires complex multiple operation transactions.

Examples of document store NoSQL databases are MongoDB, Apache CouchDB and Elasticsearch.

### Column store NoSQL database

In column-oriented NoSQL databases, data is stored in cells grouped in columns of data rather than as rows of data.

Columns are logically grouped into column families. Column families can contain a virtually unlimited number of columns that can be created at runtime or while defining the schema. Read and write is done using columns rather than rows.

Column families are groups of similar data that is usually accessed together. As an example, we often access customers' names and profile information at the same time, but not the information on their orders.

The main advantages of storing data in columns over relational DBMS are fast search/access and data aggregation. Relational databases store a single row as a continuous disk entry. Different rows are stored in different places on the disk while columnar databases store all the cells corresponding to

a column as a continuous disk entry, thus making the search/access faster.

Each column family can be compared to a container of rows in an RDBMS table, where the key identifies the row and the row consists of multiple columns. The difference is that various rows do not have to have the same columns, and columns can be added to any row at any time without having to add them to other rows.

**Use cases:** Developers mainly use column databases in:

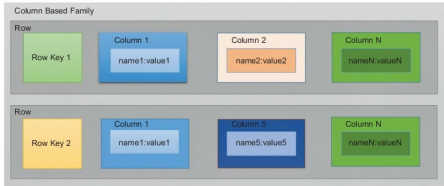


Figure 1: Column based family